MARK KESSLER

Senior Software Engineer · High-Performance Python & Scientific Computing

Frisco, TX | (563) 424-0605 | markkessler66@gmail.com
LinkedIn | Portfolio | GitHub | PyPI: phynetpy

SUMMARY

Software engineer with 6 years building high-performance Python systems. I'm the technical lead and principal architect of PhyNetPy, an open-source library on PyPI, where I handle architecture, optimization, releases, and documentation. Most of my work is on graph and statistical inference problems where the straightforward implementation is far too slow, so I spend a lot of time profiling and then fixing what the profiler actually points at. I also set technical direction, run breaking API migrations, and write the conventions other contributors build against.

TECHNICAL SKILLS

Languages
Python, Cython, JavaScript, TypeScript, C, Java, C#/.NET, SQL

Performance
Profiling (cProfile, tracemalloc), Cython extension modules, caching, vectorization, multiprocessing, CUDA/CuPy GPU offload

Data & ML
NumPy, SciPy, PyTorch, TensorFlow, scikit-learn, pandas, NetworkX

Web & Backend
ES modules, Canvas/SVG rendering, React, Flask, FastAPI, REST API design, SQLite, PostgreSQL, MongoDB

Infrastructure
GitHub Actions CI/CD, PyPI packaging and release automation, Docker, pytest, Git

AI-Assisted Dev
Repo-level assistant rules, coding standards that constrain model output, review process for generated code

EXPERIENCE

Senior Software Engineer — Technical Lead, PhyNetPy

June 2022 – Present
Rice University, Computational Systems Lab — Houston, TX (Remote)

TECHNICAL LEADERSHIP & OWNERSHIP

- Technical lead and principal architect of PhyNetPy, an open-source phylogenetic network library on PyPI. Wrote 217 of its 249 commits, covering architecture, implementation, packaging, docs, and user support.
- Led the 0.6.0 API redesign, replacing a sprawling command surface with a typed three-axis API (data × model × criterion) behind three verbs. Wrote the migration guide and compatibility matrix so users could move across a breaking release without redoing work.
- Wrote the coding standards and the repo-level rules that steer AI coding assistants — import structure, naming conventions, type hints, docstrings, validation patterns — so generated code fits the codebase instead of just looking plausible. Generated changes get reviewed against the same bar as human ones.
- Own CI/CD: a GitHub Actions matrix across Python 3.9–3.14, cross-platform smoke tests, and wheel install validation gating every release.
- Grew the test suite to **1,130 tests** at a **100% pass rate**, including memory regression, stress, and numerical cross-checks against the incumbent Java implementation.
- Built the release and docs tooling: a one-command deploy pipeline (version bump → test → build → PyPI) and a 1,100-line AST-based API reference generator.

- Onboarded and reviewed collaborators' contributions, wrote the NSF Year-1 technical report, and worked with the principal investigator on the technical roadmap.

PERFORMANCE ENGINEERING

- Shipped four Cython extension modules (graph core, pseudo-likelihood DP, coalescent DP, sequence kernels). The pseudo-likelihood kernel runs **up to 9.2×** faster than the pure-Python version and returns identical results.
- Replaced generic matrix exponentiation with closed-form and cached-eigendecomposition transition matrices across eight substitution models: **8.6–19.5×** faster than `scipy.linalg.expm`, matching it to ~1e-16.
- Rebuilt network adjacency on Cython O(1) index maps, giving **~11×** faster in-edge lookup and ~3.8× faster leaf enumeration than NetworkX on the same topologies.
- Raised search throughput **2.8×** (2,742 → 972 ms/iteration) after profiling turned up 3.7M redundant graph traversals, which I replaced with memoized lookups.
- Cut topological error **2.9×** against the incumbent PhyloNet implementation at comparable runtime, and ran **7.4×** faster (110 s vs. 818 s) on the hardest benchmark case.
- Added GPU likelihood scoring with CuPy sparse contractions and VRAM-aware batching, so out-of-memory conditions surface as recoverable errors instead of killing the run. Also added multiprocess scoring pools and parallel MCMC chains.

Software Engineering Intern / ML Research Assistant

Aug 2020 – May 2022

Rice University, Computational Systems Lab — Houston, TX

- Built Python machine-learning pipelines for large-scale data processing and model experimentation.
- Implemented tensor operations and data-encoding pipelines for high-volume numerical workloads.
- Profiled and optimized existing codebases for runtime and memory footprint.

PROJECTS

PhyNetPy — *v0.6.0 · Python 3.9–3.14 · ~41k lines across 49 modules + 4 Cython extensions*

- Framework for phylogenetic network inference, simulation, and comparison, with a typed public API, registry-based method dispatch, and explicit branch-length units that rule out a class of silent scientific errors. Sole owner across six public releases.

BioStore — *JavaScript · Flask · SQLite · Docker*

- Full-stack platform for submitting, monitoring, and visualizing long-running PhyNetPy jobs.
- Wrote the frontend by hand: ~2,500 lines of framework-free ES modules covering interactive network rendering, alignment views, a file browser, an in-browser editor, and the API client layer.
- Backend handles token auth, job execution, and lease-based file locking on SQLite (WAL) so multiple users can edit the same dataset without clobbering each other.

BioForge — *GPU-accelerated synthetic data generation*

- Python library generating synthetic DNA/RNA/protein sequences with known ground truth, for training and benchmarking ML models. Covers evolutionary models, population-genetics simulation, sequencing error models, and PyTorch-ready outputs, with runs reproducible from tracked config.

EDUCATION

Rice University — Bachelor of Arts, Computer Science

GPA 3.4